

AI-Based Traffic Risk Heatmap & Police Deployment Decision Support

Solution Overview v2 — Vikasit Bharat Hackathon, VNIT Nagpur | Problem Statement B — updated with team review feedback

“Nagpur police can't be everywhere — we built a system that tells them where the danger is right now, moves their limited officers there, and helps emergency responders reach the scene before the situation gets worse.”

The Core Idea

We simulate a traffic control room. Every major Nagpur junction gets a “danger score” out of 100, shown on a live map in green / yellow / red. The system recommends where police officers should be positioned, reacts instantly when an incident happens, and now also helps emergency vehicles (ambulance, fire brigade) find the fastest low-congestion route to the scene.

Deployment today is static — officers posted at the same spots regardless of real conditions, sometimes over-concentrated in low-risk areas while high-risk areas go under-covered. This system makes deployment dynamic, data-driven, and correctly balanced, while keeping a human in control at every step.

Where the “AI” Actually Is

The AI component is specifically the **risk-scoring model**: it takes raw inputs (traffic volume, accident history, speed, time of day, current coverage) and predicts a single risk score per junction. Everything downstream — sorting, allocation, and routing — is deterministic logic that consumes that score. This separation is deliberate: a tool directing police and emergency resources has to be explainable and auditable, not a black box.

For the hackathon build, the score is a transparent weighted formula rather than a model trained on real data, since our dataset is self-generated mock data and a formula keeps every recommendation traceable in real time. Ranking junctions is a plain sort; routing an ambulance uses a classical pathfinding algorithm (Dijkstra / A*) — also a recognized branch of AI (search-based AI), just not machine learning.

Architecture note: all scoring, allocation, and routing logic lives entirely in the backend (FastAPI). The frontend only displays what the backend sends and forwards user actions (like “simulate accident”) back as API calls — it never calculates anything itself. This keeps the “where's the AI” answer pointing at one controllable place in the system.

How It Works — Step by Step

1. We fake real Nagpur data

A list of ~30–50 real junction names (Sadar, Sitabuldi, VIP Road, Dharampeth, etc.), each with realistic but simulated numbers: traffic volume, past accident count, average speed, and time of day.

2. Every junction gets a danger score

A weighted formula combines those numbers into one score from 0 to 100. Higher traffic, more accidents, and erratic speeds push the score up; more police already posted there pulls it down. Fully explainable — every score can be traced back to the numbers that produced it.

3. The score becomes a color on the map

0–40 = green (low risk), 40–70 = yellow (medium), 70–100 = red (high). Rendered as an interactive heatmap over an actual Nagpur map using Leaflet.js.

4. We rank junctions and recommend deployment

Junctions are sorted highest to lowest risk. Given a limited number of officers, the system assigns them to the highest-risk junctions first. This same logic naturally fixes over-policing: officers sitting in a low-risk area (e.g. Bansi Nagar) get reassigned toward a higher-risk, under-covered area (e.g. Lokmanya Nagar) instead of staying put by habit.

5. We simulate a live incident

A button triggers “Accident at Junction J3.” That junction's score jumps immediately, the map updates, and allocation re-runs. On a confirmed accident, the system recommends splitting the response: part of the redeployed officers manage the accident scene, the rest manage traffic flow around it so congestion doesn't compound the emergency.

6. We route emergency vehicles around the congestion

Once an incident is logged, an ambulance or fire brigade unit can query the system for the fastest route to that junction that avoids currently congested roads. This uses a shortest-path algorithm (Dijkstra/A*) over a small graph of our demo junctions, with each road segment weighted by its current congestion score.

7. We let a human override it

Every recommendation has a manual “reassign” control — the system supports police judgement, it doesn't replace it. Every override is logged for an audit trail.

8. We show before vs. after

A toggle switches between the current fixed deployment and the recommended deployment — the fastest way to prove the system adds value, including fixing the over/under-policing imbalance.

Extensions Added After Team Review

Idea	What It Means	How We Handle It
Emergency services access (ambulance, fire brigade)	Not just police — other responders view the heatmap to pick the least congested route	Add a routing endpoint using Dijkstra/A* over the demo junction graph; scoped as one team member's dedicated feature, not folded into core scoring work
Fixing over/under-policing (e.g. Bansi Nagar vs. Lokmanya Nagar)	Some areas have more officers than their risk justifies, others too few	Already solved by the existing allocation algorithm — no new build needed, just needs to be called out explicitly in the demo and pitch
Split incident response + ambulance guidance	On accident, redeploy officers to both manage the scene and control surrounding traffic; guide the ambulance in	Extend the incident-simulation logic to split redeployed officers by role; connect ambulance routing to the same incident trigger

Scope warning: these are genuinely good additions, but each is a separate subsystem. Treat routing as a stretch goal owned by one person, built only after the core 9-item checklist (below) is working end-to-end.

Mapping to the Official Expected Solution

#	Required Item	Our Approach
i	Traffic-risk scoring model	Weighted formula run on mock data for ~30–50 junctions
ii	Interactive heatmap	Leaflet.js map, colour-coded by risk bucket
iii	Ranked list of locations	Junctions sorted by score, shown in a sidebar table
iv	Personnel-allocation algorithm	Greedy allocation to top-N riskiest junctions; corrects over/under-policing
v	Dynamic redeployment (simulated incident)	“Simulate accident” button re-scores and re-allocates live, split by role
vi	Unmanned high-risk locations	Auto-flag: risk = high AND assigned officers = 0
vii	Explainable recommendations + override	Score breakdown shown per junction; manual reassign + log
viii	Before/after comparison	Toggle: baseline fixed deployment vs. AI-recommended deployment
ix	Control-room dashboard	Single page: map + ranked list + allocation panel + incident trigger
stretch	Emergency-service routing	Dijkstra/A* shortest low-congestion path for ambulance/fire brigade

Tech Stack

- **Backend:** Python + FastAPI — scoring formula, allocation, incident simulation, routing endpoints
- **Frontend:** React (or plain HTML/JS) + Leaflet.js for the interactive map
- **Routing:** Dijkstra / A* over a small hand-built graph of demo junctions, edge weights = congestion
- **Data:** Simulated / mocked junction dataset (CSV or JSON), no external APIs required
- **Hosting for demo:** Local or a quick free-tier deploy (Render / Vercel) — not a priority over a working local demo

Team Roles (5 Members)

Member	Responsibility
You (AI-assisted backend)	FastAPI endpoints (/junctions, /risk-scores, /simulate-incident, /allocate, /override), scoring formula, allocation algorithm
Frontend developer	Leaflet map + dashboard layout, wiring to backend API
Member 3	Mock data generation: believable Nagpur junction dataset + a small demo road graph for routing
Member 4	Before/after comparison logic + split incident-response allocation (police/traffic control)
Member 5	Stretch goal: emergency-service routing endpoint (Dijkstra/A*), plus pitch deck, demo script, and rehearsal if routing isn't reached

Judge Q&A — Rehearsed Answers

Q: “Where's the AI in this?”

“The AI is the risk-scoring model — it takes live traffic, accident history, and coverage data for each junction and predicts a risk score. Everything downstream, like sorting, allocation, and routing, runs on deterministic logic that consumes that score. We did this deliberately: a system recommending police and emergency deployment has to be explainable, not a black box.”

Q: “Is it actually a trained model, or just a formula?”

“For the hackathon we implemented it as a weighted scoring function so every recommendation is fully explainable in real time. The same model can be swapped for one trained on real historical city data — we scoped that as the production upgrade, not the demo.”

Q: “How do you know an accident happened?”

“For the demo, it's manually triggered to simulate a real report. In production this would come from crowdsourced reports, CCTV-based detection, or a police dispatch integration — we've scoped that as a roadmap item, not built it, since it needs infrastructure access we don't have.”

Q: “Is this for the public or only for police?”

“Only for authorized traffic and emergency personnel — police, control room operators, and now ambulance/fire brigade units. This is intentional: showing coverage gaps publicly would be a safety risk, not a feature.”

Scope discipline for the 24 hours: build the 9-item core checklist end-to-end first. Emergency-service routing is a stretch goal — attempt it only once the core dashboard works and demos cleanly. Real-time sensor integration, authentication/RBAC, and audit systems remain future-work roadmap items, not hackathon builds.